Unix & Shell Programming

Module 6 – grep & awk (Pattern Searching)

Question & Answer Notes

BCA Semester 6 · MAKAUT

Q1. What is grep? Explain basic pattern searching with examples.

grep stands for **Global Regular Expression Print**. It is a Unix command used to search for a pattern (text or regular expression) inside files or input. It prints all lines that match the given pattern.

Syntax:

```
grep [options] "pattern" filename
```

Examples:

```
# Search for the word "hello" in file.txt
grep "hello" file.txt

# Search for "root" in /etc/passwd
grep "root" /etc/passwd

# Search for pattern in multiple files
grep "error" file1.txt file2.txt
```

grep reads the file line by line and prints only those lines where the pattern is found.

Q2. Explain advanced grep options.

grep has many useful options:

Option	Meaning
`-i`	Ignore case (case-insensitive search)
`-v`	Invert match (print lines that do NOT match)
`-c`	Count number of matching lines
`-n`	Show line numbers
`-l`	Print only file names that contain the match
`-r`	Recursive search in directories
`-w`	Match whole word only
`-e`	Search for multiple patterns

Examples:

```
grep -i "hello" file.txt        # case-insensitive
grep -v "error" file.txt        # lines without "error"
```

```
grep -c "warning" file.txt      # count matches
grep -n "main" program.c        # show line numbers
grep -r "TODO" /home/student/   # recursive search
grep -w "is" file.txt           # match whole word only
grep -e "cat" -e "dog" file.txt  # multiple patterns
```

Q3. What is awk? Explain its basic structure.

awk is a powerful text-processing tool in Unix. It processes files line by line and can search, filter, calculate, and format output. The name comes from its creators: **A**ho, **W**einberger, and **K**ernighan.

Basic Syntax:

```
awk 'pattern { action }' filename
```

- **pattern** – a condition to match (optional)
- **action** – what to do when pattern matches
- awk reads the file line by line
- Each line is split into fields: `$1`, `$2`, ..., `$NF`
- `$0` = entire line, `NF` = number of fields, `NR` = line number

Example:

```
# Print first field of each line
awk '{ print $1 }' file.txt

# Print lines where second field is greater than 50
awk '$2 > 50 { print $0 }' file.txt
```

Q4. Explain the use of print and printf in awk.

print – prints output with a newline at the end automatically.

printf – prints formatted output (like C's printf), no automatic newline.

Examples of print:

```
awk '{ print $1, $2 }' file.txt      # prints field1 and field2
awk '{ print NR, $0 }' file.txt      # prints line number and
   full line
```

Examples of printf:

```
awk '{ printf "Name: %s, Age: %d\n", $1, $2 }' file.txt
awk '{ printf "%-10s %5d\n", $1, $2 }' file.txt
```

Format specifiers:

Specifier	Meaning
%s	String
%d	Integer
%f	Float
%e	Scientific notation
\n	Newline
\t	Tab

Q5. Explain awk comparison operators.

awk supports standard comparison operators used in pattern matching:

Operator	Meaning	Example
==	Equal to	$2 == 100
!=	Not equal to	$1 != "admin"
>	Greater than	$3 > 50
<	Less than	$3 < 10
>=	Greater than or equal	$2 >= 60
<=	Less than or equal	$2 <= 40
~	Match regex	$1 ~ /abc/
!~	Not match regex	$1 !~ /abc/

Examples:

```
# Print lines where marks > 40
awk '$2 > 40 { print $1, "Pass" }' marks.txt

# Print lines where name is "Ram"
awk '$1 == "Ram" { print $0 }' file.txt

# Print lines where name contains "an"
awk '$1 ~ /an/ { print $0 }' file.txt
```

Q6. Explain awk arithmetic operators with examples.

awk can perform arithmetic operations on field values:

Operator	Meaning	Example
+	Addition	$2 + $3
-	Subtraction	$2 - $3
*	Multiplication	$2 * $3
/	Division	$2 / $3
%	Modulus	$2 % $3
^	Exponent	$2 ^ 2

Examples:

```
# Add field 2 and field 3 and print
awk '{ print $1, $2 + $3 }' file.txt
```

```
# Calculate average of two fields
awk '{ avg = ($2 + $3) / 2; print $1, avg }' file.txt

# Calculate total marks
awk '{ total += $2 } END { print "Total:", total }' marks.txt
```

Q7. Explain BEGIN and END sections in awk.

awk has two special sections:

- **BEGIN** – executed *before* reading any input line. Used for initialization, printing headers, setting variables.
- **END** – executed *after* all input lines are processed. Used for final calculations, printing summary.

Syntax:

```
awk 'BEGIN { ... } { ... } END { ... }' file.txt
```

Example:

```
awk '
BEGIN {
    print "Name      Marks"
    print "-----------------"
}
{
    print $1, $2
    total += $2
    count++
}
END {
    print "-----------------"
    printf "Average: %.2f\n", total/count
}
' marks.txt
```

BEGIN and END blocks are optional. You can use any combination – only BEGIN, only END, or both.

Q8. Explain the use of if–else in awk.

awk supports `if-else` statements inside the action block, just like in C.

Syntax:

```
awk '{ if (condition) { action1 } else { action2 } }' file.txt
```

Example 1 – Pass/Fail check:

```
awk '{
    if ($2 >= 40)
        print $1, "PASS"
    else
        print $1, "FAIL"
}' marks.txt
```

Example 2 – Grade calculation:

```
awk '{
    if ($2 >= 80)
        grade = "A"
    else if ($2 >= 60)
        grade = "B"
    else if ($2 >= 40)
        grade = "C"
    else
        grade = "F"
    print $1, grade
}' marks.txt
```

Q9. Explain built-in variables FS and OFS.

FS (Field Separator):

- FS is the input field separator used by awk to split each line into fields.
- By default, FS is a space or tab.
- We can change FS to any character like comma, colon, etc.

OFS (Output Field Separator):

- OFS is the separator used between fields when printing output.
- By default, OFS is a space.

Examples:

```
# Read CSV file (comma separated)
awk -F',' '{ print $1, $2 }' data.csv

# Set FS in BEGIN block
awk 'BEGIN { FS=":" } { print $1 }' /etc/passwd

# Change OFS to comma
awk 'BEGIN { OFS="," } { print $1, $2, $3 }' file.txt

# Change both FS and OFS
awk 'BEGIN { FS=":"; OFS="|" } { print $1, $3 }' /etc/passwd
```

`-F` option is a shortcut to set FS from command line.

Q10. Explain awk string functions.

awk has several built-in string functions:

Function	Description
`length(s)`	Returns length of string `s`
`substr(s, m, n)`	Returns substring of `s` from position `m`, length `n`
`index(s, t)`	Returns position of string `t` in `s`
`split(s, a, fs)`	Splits string `s` into array `a` using separator `fs`
`toupper(s)`	Converts string to uppercase
`tolower(s)`	Converts string to lowercase
`gsub(r, s)`	Replace all occurrences of regex `r` with `s`
`sub(r, s)`	Replace first occurrence of regex `r` with `s`

Examples:

```
awk '{ print length($1) }' file.txt           # length of field 1
awk '{ print substr($1, 1, 3) }' file.txt     # first 3 characters
awk '{ print toupper($0) }' file.txt          # convert to uppercase
awk '{ print index($1, "an") }' file.txt      # position of "an"
awk '{ gsub(/error/, "ERROR"); print }' log.txt  # replace all
```

Q11. Explain awk arithmetic functions.

awk supports mathematical functions from the standard C library:

Function	Description
`int(x)`	Returns integer part of x (truncates)
`sqrt(x)`	Square root of x
`log(x)`	Natural logarithm of x
`exp(x)`	e raised to the power x
`sin(x)`	Sine of x (in radians)
`cos(x)`	Cosine of x (in radians)
`atan2(y,x)`	Arctangent of y/x
`rand()`	Random number between 0 and 1
`srand(x)`	Sets seed for random number generator

Examples:

```
awk '{ print sqrt($1) }' file.txt        # square root
awk '{ print int($1 / 3) }' file.txt     # integer division
awk 'BEGIN { print sin(3.14/2) }'        # sin(90 degrees approx)
awk 'BEGIN { srand(); print rand() }'    # random number
awk '{ print log($1) }' file.txt         # natural log
```

Q12. Write an awk script using loops.

awk supports `for`, `while`, and `do-while` loops.

Example 1 – for loop (print multiplication table):

```
awk 'BEGIN {
    n = 5
    for (i = 1; i <= 10; i++) {
        print n, "x", i, "=", n * i
    }
}'
```

Example 2 – while loop (sum of numbers 1 to 10):

```
awk 'BEGIN {
    i = 1
    sum = 0
    while (i <= 10) {
        sum += i
        i++
    }
    print "Sum =", sum
}'
```

Example 3 – for loop to print all fields of each line:

```
awk '{
    for (i = 1; i <= NF; i++) {
        printf "Field %d: %s\n", i, $i
    }
}' file.txt
```

`NF` is a built-in variable that holds the number of fields in the current line.

Q13. Explain substitution and search functions in awk.

awk provides functions to search and replace patterns within strings:

1. sub(regex, replacement [, target])

- Replaces the *first* occurrence of regex with replacement.
- If target not given, it works on `$0` (whole line).

```
# Replace first "cat" with "dog" in each line
awk '{ sub(/cat/, "dog"); print }' file.txt

# Replace in a specific field
awk '{ sub(/error/, "ERROR", $2); print }' file.txt
```

2. gsub(regex, replacement [, target])

- Replaces *all* occurrences of regex with replacement.

```
# Replace all spaces with underscore
awk '{ gsub(/ /, "_"); print }' file.txt

# Replace all digits with X
awk '{ gsub(/[0-9]/, "X"); print }' file.txt
```

3. match(string, regex)

- Searches for regex in string.
- Returns the position (starting from 1) if found, 0 if not.
- Sets built-in variables `RSTART` and `RLENGTH`.

```
# Find position of digits in $0
awk '{
    pos = match($0, /[0-9]+/)
    if (pos > 0)
        print "Found at position:", pos, "Length:", RLENGTH
}' file.txt
```

Difference between sub and gsub: `sub` replaces only the first match; `gsub` replaces all matches. Use `gsub` when you want to replace every occurrence in the line.

End of Module 6 Notes